

UNIT – 1
ASSESSMENT – 2
PYTHON CODE

NIHARIKA PREEJESH
192524506

1. # Doctor Shift Scheduling using Backtracking

```
doctors = ["D1", "D2", "D3"]
```

```
shifts = ["Morning", "Afternoon", "Night"]
```

```
assignment = {}
```

```
def is_valid(doctor, shift):
```

```
    if doctor == "D1" and shift == "Night":
```

```
        return False
```

```
    if doctor == "D3" and shift == "Morning":
```

```
        return False
```

```
    if shift in assignment.values():
```

```
        return False
```

```
    return True
```

```
def backtrack(index):

    if index == len(doctors):

        if shifts.index(assignment["D2"]) <
shifts.index(assignment["D3"]):

            return True

        return False

    doctor = doctors[index]

    for shift in shifts:

        if is_valid(doctor, shift):

            assignment[doctor] = shift

            if backtrack(index + 1):

                return True

            del assignment[doctor]

    return False
```

```
if backtrack(0):
```

```
    print("Valid Schedule\n")
```

```
    for doctor in doctors:
```

```
        print(doctor, "->", assignment[doctor])
```

```
else:
```

```
    print("No valid assignment found.")
```

2. # Robot Path Planning using Breadth-First Search (BFS)

```
from collections import deque
```

```
grid = [  
    ['S', '.', 'X', '.', '.'],  
    ['.', '.', 'X', '.', '.'],  
    ['.', '.', '.', 'X', '.'],  
    ['X', 'X', '.', '.', '.'],  
    ['.', '.', '.', 'X', 'G']  
]
```

```
rows = len(grid)
```

```
cols = len(grid[0])
```

```
start = (0, 0)
```

```
goal = (4, 4)
```

```
moves = [(-1,0),(1,0),(0,-1),(0,1)]
```

```
queue = deque()
```

```
queue.append((start,[start]))
```

```
visited = set()
```

```
visited.add(start)
```

```
while queue:
```

```
    current, path = queue.popleft()
```

```
    if current == goal:
```

```
        print("Shortest Path:")
```

```
        print(path)
```

```
        print("Total Cost:", len(path)-1)
```

```
        break
```

for dx,dy in moves:

nx = current[0]+dx

ny = current[1]+dy

if $0 \leq nx < \text{rows}$ and $0 \leq ny < \text{cols}$:

if grid[nx][ny] != 'X' and (nx,ny) not in visited:

visited.add((nx,ny))

queue.append(((nx,ny), path+[(nx,ny)]))

3. #Autonomous Rescue Robot using Online Search / Uniform Cost Search

from queue import PriorityQueue

graph = {

'S': [('A',1), ('B',3)],

'A': [('C',1)],

'B': [('D',2)],

'C': [('G',2)],

'D': [('G',1)],

```
'G': []  
}
```

```
def uniform_cost_search(start, goal):
```

```
    pq = PriorityQueue()  
    pq.put((0,start,[start]))
```

```
    visited = set()
```

```
    while not pq.empty():
```

```
        cost,node,path = pq.get()
```

```
        if node == goal:  
            return cost,path
```

```
        if node not in visited:
```

```
            visited.add(node)
```

```
            for neighbour,edge_cost in graph[node]:
```

```
                if neighbour not in visited:
```

```
    pq.put((cost+edge_cost,  
            neighbour,  
            path+[neighbour]))
```

```
cost,path = uniform_cost_search('S','G')
```

```
print("Optimal Path:")
```

```
print(" -> ".join(path))
```

```
print("Total Cost:",cost)
```